

Slide set # 10

Object-Oriented Programming (3)

Introduction to Programming for Engineers

Dr. Inon Zuckerman, Dr. Chen Hajaj

Industrial Engineering and Management

חלק מהשקפים נלקחו מקורסים דומים באוניברסיטת תל אביב ובר אילן ולהם התודה.

Object Oriented Programming

- Classes represent new data types
- Classes have attributes and methods
- Objects are instances of a class
- Design program by objects interaction
- Encapsulation
- Polymorphism
- Operator overloading

Example: Pets

We'll implement a toy example – pets,
which handles different **types** of pets.

Dogs

A dog has:

1. Name
2. Year of Birth
3. Color
4. Playmates

A dog can:

1. Make noise (bark)
2. Play with others:
 1. Add playmate
 2. Remove playmate



Class Dog - Design

- Attributes: name, birth year, color
- Constructor: given name, year and color.
- Default playmates: empty list.
- Methods:
 1. *play_with(self, other)*: add *other* to the list of playmates in case it is not already there.
 2. *stop_play(self, other)*: checks if *other* is in playmates, removes it if true.
 3. *make_noise(self)*: returns dog noises
 4. *__repr__(self)*: print attributes

Class Dog - Implementation

```
class Dog:
    def __init__(self, name, birth_year, color):
        self.name, self.birth_year, self.color = name, birth_year, color
        self.playmates = []

    def make_noise(self):
        return "woof! woof!"

    def play_with(self, other):
        if other not in self.playmates:
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        return result
```

Utility Function: Getting Current Year

```
import datetime

def get_current_year():
    return datetime.datetime.now().year
```

Class Dog – Run Example

```
>>> doge = Dog("doge", 2000, "black")
>>> doggo = Dog("doggo", 2010, "golden")
>>> doge.play_with(doggo)
>>> print(doge)
name: doge
age: 21
color: black
plays with: [
    doggo
]
```


Shibas

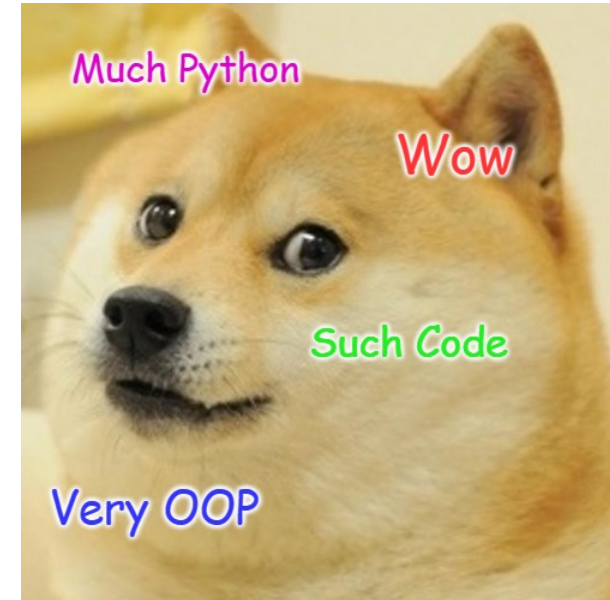
Shibas are a type of dog breed.

A shiba has:

1. Name
2. Year of Birth
3. Color – **Always golden**
4. Playmates

A shiba can:

1. Make noise
2. Play with others: Add/Remove playmates
3. **Wag Tail**



Class Shiba - Design

- Attributes: name, birth year, color
- Constructor: given name, year ~~and color~~.
- Default playmates: empty list.
- Default tail wagging: False.
- Methods:
 1. *play_with(self, other)*: add *other* to the list of playmates in case it is not already there.
 2. *stop_play(self, other)*: checks if *other* is in playmates, removes it if true.
 3. *make_noise(self)*: returns dog noises
 4. *wag_tail(self)*: set tail wagging to True
 5. *__repr__(self)*: pretty print attributes

Class Shiba - Implementation

```
class Shiba:
    def __init__(self, name, birth_year):
        self.name, self.birth_year = name, birth_year
        self.color = "golden"
        self.is_tail_wagging = False
        self.playmates = []

    def make_noise(self):
        return "woof! woof!"

    def wag_tail(self):
        self.is_tail_wagging = True

    def play_with(self, other):
        if other not in self.playmates:
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        if self.is_tail_wagging:
            result += f"\n{self.name} is wagging its tail"
        return result
```

Problem

Our Dog and Shiba classes looks similar and have a lot of duplicated code.

```
class Shiba:
    def __init__(self, name, birth_year):
        self.name, self.birth_year = name, birth_year
        self.color = "golden"
        self.is_tail_wagging = False
        self.playmates = []

    def make_noise(self):
        return "woof! woof!"

    def wag_tail(self):
        self.is_tail_wagging = True

    def play_with(self, other):
        if other not in self.playmates:
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        if self.is_tail_wagging:
            result += f"\n{self.name} is wagging its tail"
        return result
```

```
class Dog:
    def __init__(self, name, birth_year, color):
        self.name, self.birth_year = name, birth_year
        self.color = color
        self.playmates = []

    def make_noise(self):
        return "woof! woof!"

    def play_with(self, other):
        if other not in self.playmates:
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        return result
```

Solution: Encapsulation

Let's reuse code from class Dog inside class Shiba.

- Avoids duplicated code
- Reuses similar code
- Solves our problem

But how do we do that?

We can try creating a Dog object inside the constructor of Shiba.

- That's one way to do it...

Class Shiba - Encapsulation

```
class Shiba:
    def __init__(self, name, birth_year):
        self.dog = Dog(name, birth_year, "golden")
        self.is_tail_wagging = False

    def make_noise(self):
        return self.dog.make_noise()

    def wag_tail(self):
        self.is_tail_wagging = True

    def play_with(self, other):
        self.dog.play_with(other)

    def stop_play(self, other):
        self.dog.stop_play(other)

    def __repr__(self):
        result = str(self.dog)
        if self.is_tail_wagging:
            result += f"\n{self.name} is wagging its tail"
        return result
```

Solution: Encapsulation

We still must call Dog methods from inside Shiba Methods.

- What if we have a lot of methods?
- Can we do better?

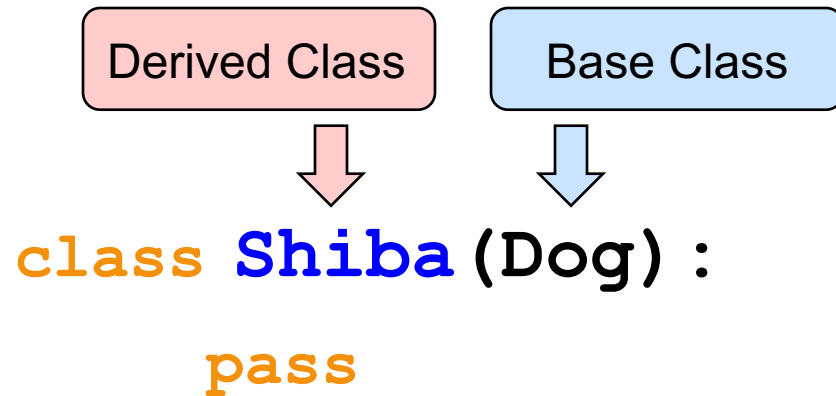
Class Inheritance

Class B inherits from class A:

- All methods from Class A are also in B
- All class attributes from Class A are also in B
- Built-in function `isinstance()` supports inheritance
- Reuses code efficiently

Inheritance is commonly used to reflect *is-a* relationship between classes: B is a subtype of A.

Python Inheritance



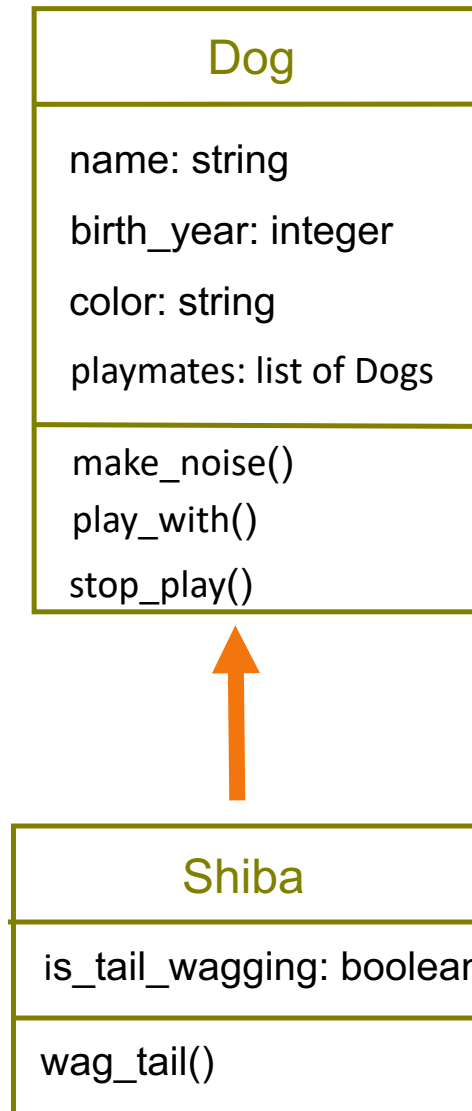
- A Shiba is a specific type of a Dog.
- Class Dog is said to be a **base** class
- Class Shiba is said to be a **derived** class

Class Inheritance

```
class Shiba(Dog):  
    def __init__(self, name, birth_year):  
        super().__init__(name, birth_year, "golden")  
        self.is_tail_wagging = False  
  
    def wag_tail(self):  
        self.is_tail_wagging = True  
  
    def __repr__(self):  
        result = super().__repr__()  
        if self.is_tail_wagging:  
            result += f"\n{self.name} is wagging its tail"  
        return result
```

- Use the keyword **super()** to call parent methods, including the constructor.
- Make necessary changes to methods.
- More importantly, reuses code perfectly.

Class Diagram



Overriding Methods

- In order to add the *wag_tail* method and related attribute we redefine the methods *__init__* and *__repr__* in Shiba class.
- The new implementation overrides the *__init__* and *__repr__* methods in class Dog.
- When *__init__* or *__repr__* is called, if the object is a Shiba, it will use these methods instead.

Overriding Methods - Run

```
>>> shiba = Shiba("shib", 2003)
>>> shiba.is_tail_wagging
False
>>> print(shiba)
name: shib
age: 16
color: golden
plays with: [
]
>>> shiba.wag_tail()
>>> print(shiba)
name: shib
age: 16
color: golden
plays with: [
]
shib is wagging its tail
```

Cats

A cat has:

1. Name
2. Year of Birth
3. Color
4. Playmates

A cat can:

1. Make noise
2. Play with others:
 1. Add playmate
 2. Remove playmate



Class Cat - Design

- Attributes: name, birth year, color
- Constructor: given name, year and color.
- Methods:
 1. *play_with(self, other)*: add *other* to the list of playmates in case it is not already there **and** *other* is **also a Cat** (cats are snobs).
 2. *stop_play(self, other)*: checks if *other* is in playmates, removes it if true.
 3. *make_noise(self)*: returns cat noises
 4. *__repr__(self)*: Pretty print attributes

Class Cat - Implementation

```
class Cat:
    def __init__(self, name, birth_year, color):
        self.name, self.birth_year, self.color = name, birth_year, color
        self.playmates = []

    def make_noise(self):
        return "meow! meow!"

    def play_with(self, other):
        if other not in self.playmates and isinstance(other, Cat):
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        return result
```


Inheritance: Cats and Dogs

We said that inheritance reflects an *is-a* relationship between derived and base class.

- Most code is duplicated between Cat and Dog.
- Cat **is not** a Dog
- Dog **is not** a Cat
- **Solution:** build a base class for both.
 - A good name would be Pet or Animal

Class Pet - Implementation

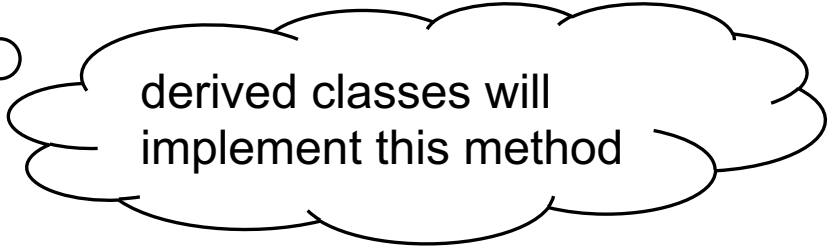
```
class Pet:
    def __init__(self, name, birth_year, color):
        self.name, self.birth_year, self.color = name, birth_year, color
        self.playmates = []

    def make_noise(self):
        raise NotImplementedError

    def play_with(self, other):
        if other not in self.playmates:
            self.playmates.append(other)

    def stop_play(self, other):
        if other in self.playmates:
            self.playmates.remove(other)

    def __repr__(self):
        age = get_current_year() - self.birth_year
        result = f"name: {self.name}\nage: {age}\ncolor: {self.color}\n"
        result += "plays with: [\n"
        for playmate in self.playmates:
            result += f"\t {playmate.name}\n"
        result += "]"
        return result
```



derived classes will
implement this method

Classes Dog, Cat and Shiba

```
class Dog(Pet):
    def make_noise(self):
        return "woof! woof!"

class Cat(Pet):
    def make_noise(self):
        return "meow! meow!"

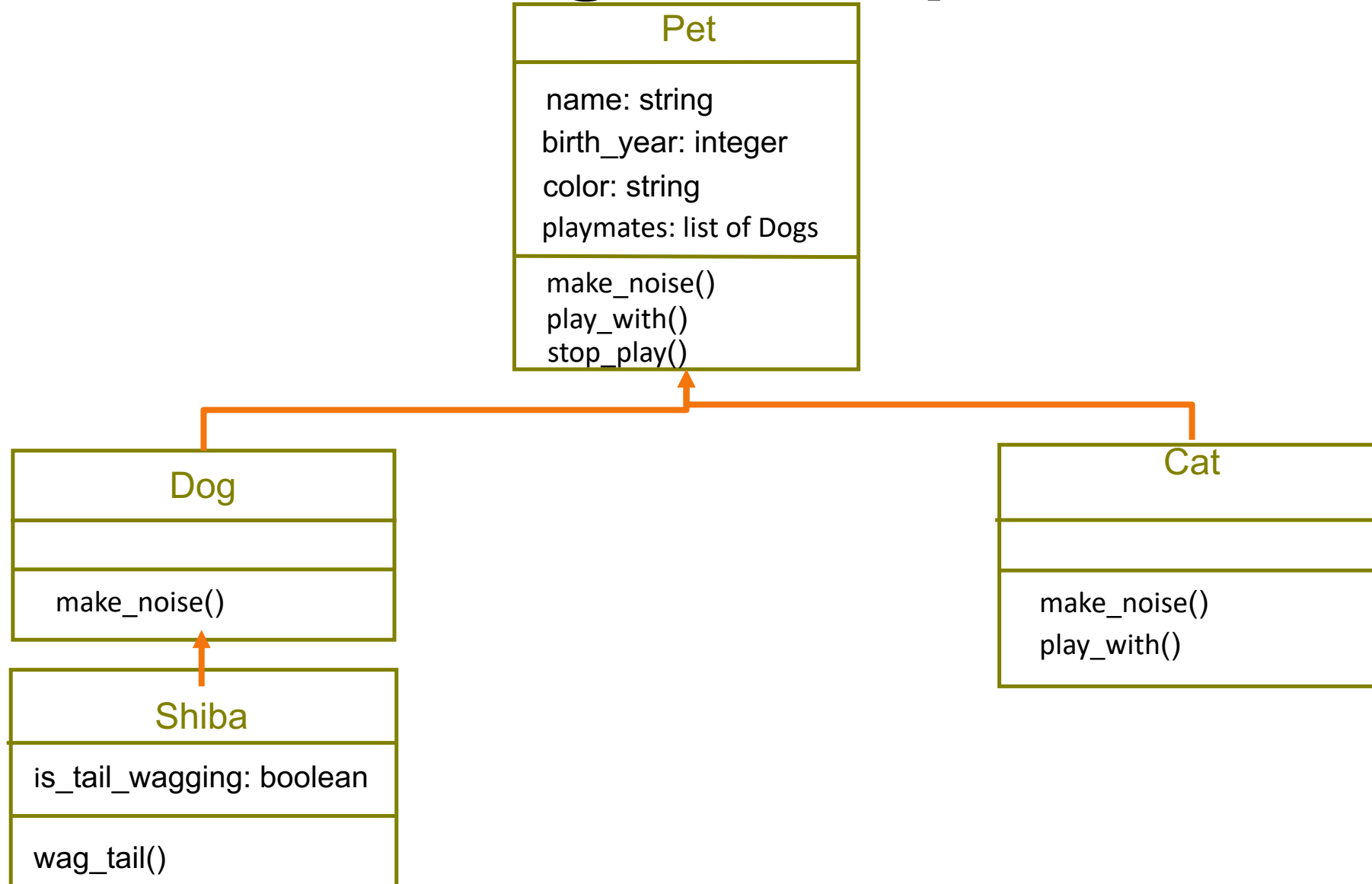
    def play_with(self, other):
        if other not in self.playmates and isinstance(other, Cat):
            self.playmates.append(other)

class Shiba(Dog):
    def __init__(self, name, birth_year):
        super().__init__(name, birth_year, "golden")
        self.is_tail_wagging = False

    def wag_tail(self):
        self.is_tail_wagging = True

    def __repr__(self):
        result = super().__repr__()
        if self.is_tail_wagging:
            result += f"\n{self.name} is wagging its tail"
        return result
```

Class Diagram - Updated



isinstance and isinstance

- ***isinstance*** checks the type of an instance
- ***issubclass*** checks class inheritance

```
>>> doggy = Dog("doge", 2010, "brown")
>>> isinstance(a, Dog)
True
>>> isinstance(a, Shiba)
False
>>> inu_shiba = Shiba("shibby", 2019)
>>> isinstance(inu_shiba, Pet)
True
>>> isinstance(inu_shiba, Shiba)
True
>>> isinstance(inu_shiba, Cat)
False
>>> issubclass(Shiba, Pet)
True
```

Encapsulation Violation

- Separate functionality from implementation!
- The user should not be aware of internal changes of the class implementation.
- What if the name of the attribute is changed?
- What if the attribute is changed to dictionary?
- Solution: access the attributes using a getter method (`get_playmates()` method will return the list of playmates)

Software Design

It is very important to design the classes in our software in an intuitive and efficient manner. Sometimes, this requires adding objects that do not necessarily exist in real life.

University Life

Let's look into Python course. We want to write classes that will enable us to write code that relates to the daily life of the course.

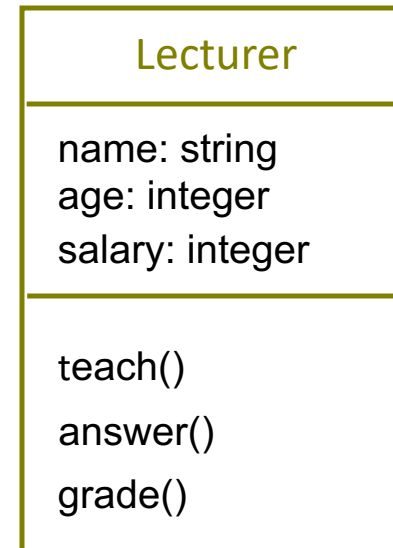
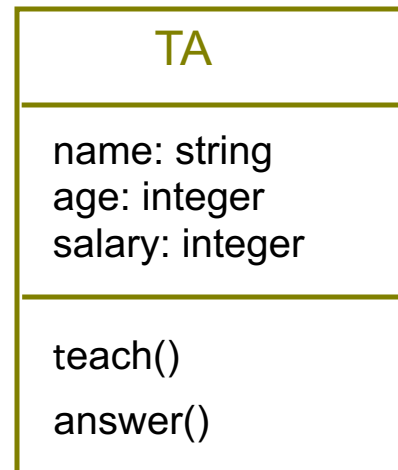
Python Course Classes

- Student
- Lecturer
- Teaching Assistant
- Assignment
- More...

Classes design

- **Student:**
name, age, grade.
study, ask, do homework.
- **Teaching Assistant:**
name, age, salary.
teach, answer.
- **Lecturer:**
name, age, salary.
teach, answer, grade.

Class Diagram (I)

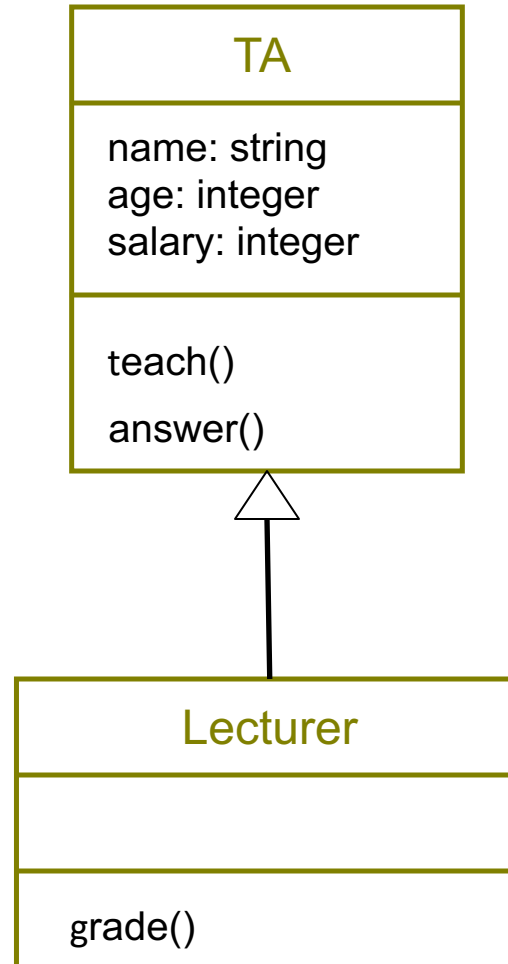


CODE DUPLICATION!

Class Diagram (II)



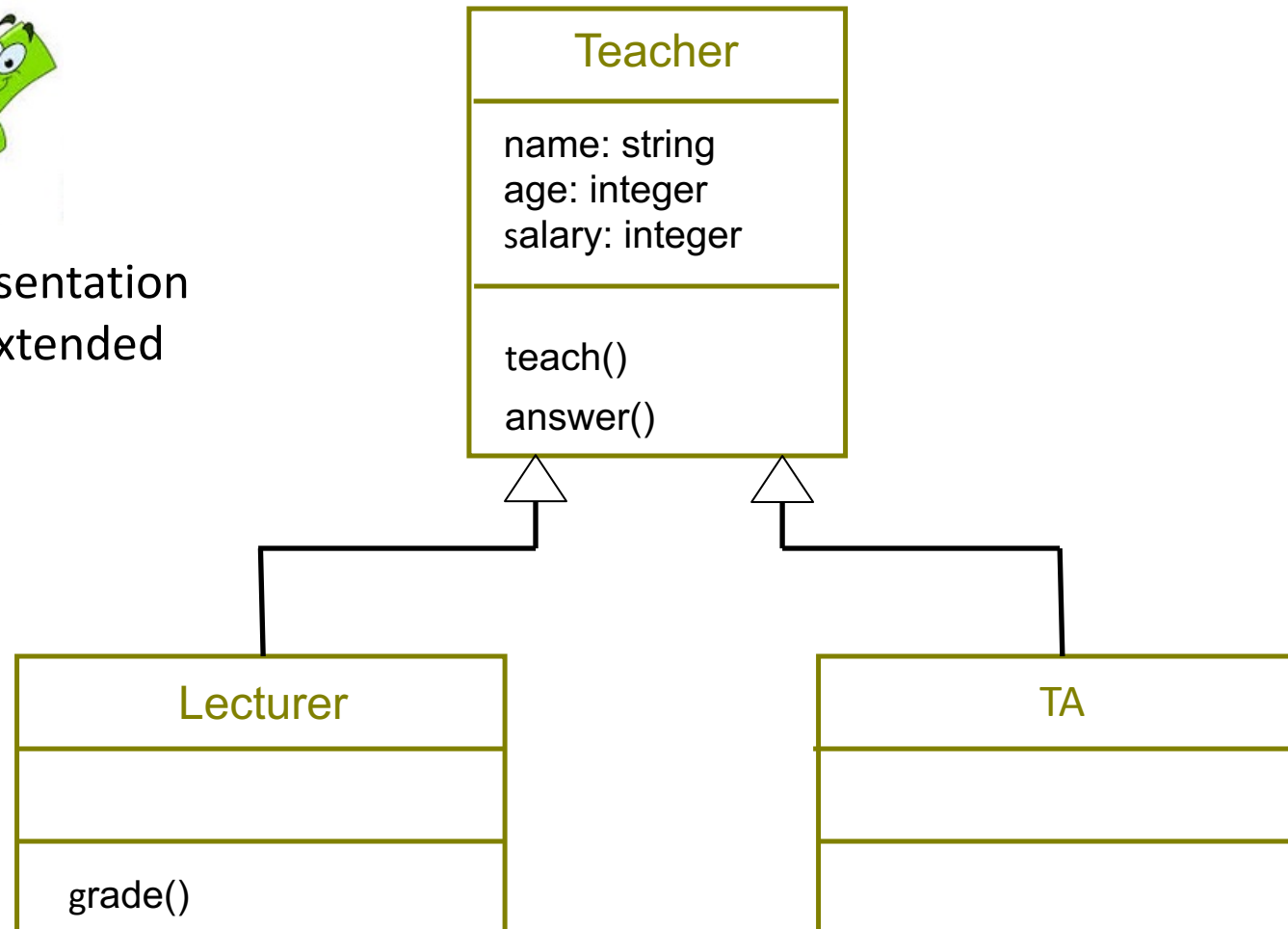
NOT IS-A RELATIONSHIP!



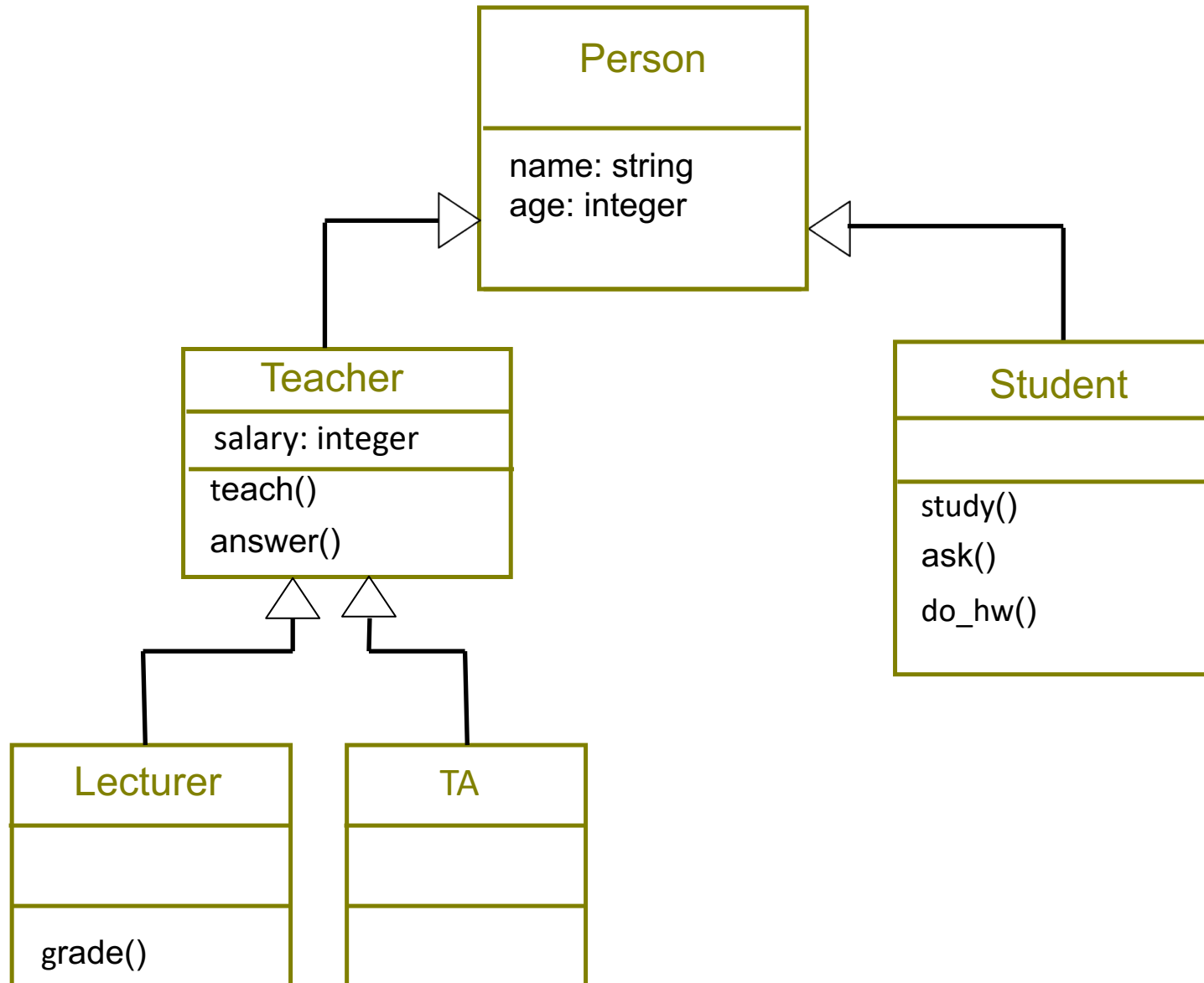
Class Diagram (II)



1. Accurate representation
2. Can be easily extended



Class Diagram (III)



Let's work

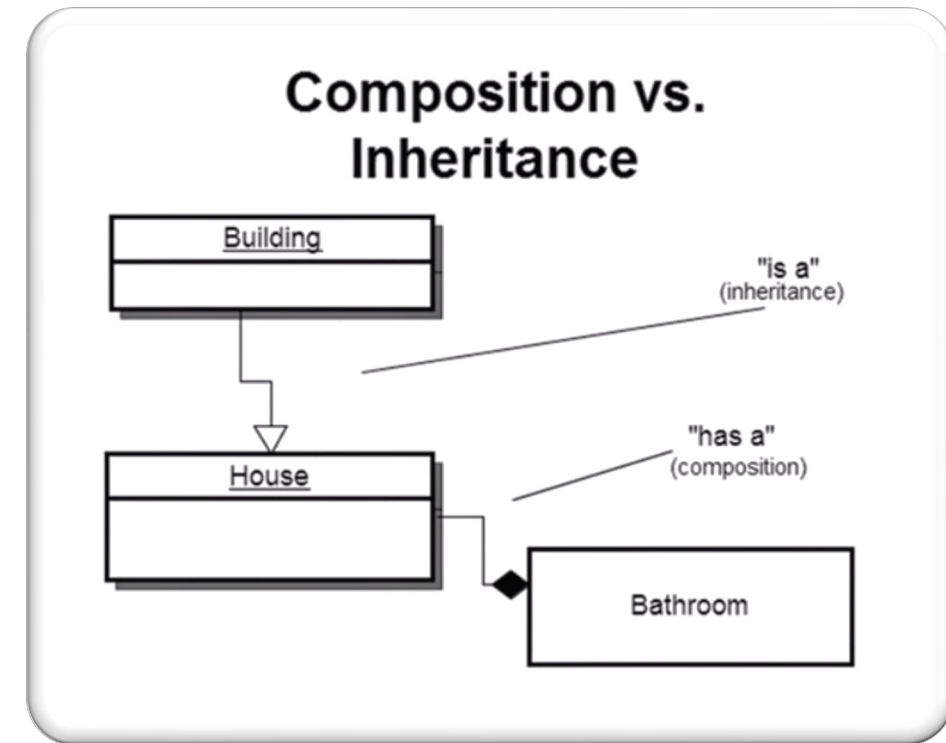
- The university opened a preparatory program (מכינה)
- Students in the program are identical to other students in most of the attributes
- The only difference is that it have two additional grades: Math and English (in addition to the grade average, each student has).
- Create the Student class and the PrepStudent class

Keep on working

- The university holds one list of lecturers, students and TAs
- The dean wants to publish the names of the students under the preparatory program whose English grades is higher than 85, and Math grade is higher than 90
- How would you do that?

Inheritance and composition

- **Inheritance** means that two objects are related by means of an *Is-A* type of relationship.
- On the other hand, **composition** means that two objects are related by means of a *Has-A* type of relationship



Inheritance and composition (example)

```
class Engine():
    def start(self):
        print("engine started")
    def stop(self):
        print("engine stopped")
class ElectricEngine(Engine): # Is-A Engine
    pass
class V8Engine(Engine): # Is-A Engine
    pass
```

```
class Car():
    def __init__(self,engine):
        self.engine = engine # Has-A Engine
    def start(self):
        print('Starting engine {0} for car {1}'.format(self.engine.__class__.__name__,self.__class__.__name__))
        self.engine.start()
    def stop(self):
        self.engine.stop()
```

Inheritance and composition (Cont')

```
class RaceCar(Car): # Is-A Car
    pass
class CityCar(Car): # Is-A Car
    pass
class F1Car(RaceCar): # Is-A RaceCar and also Is-A Car
    pass
```

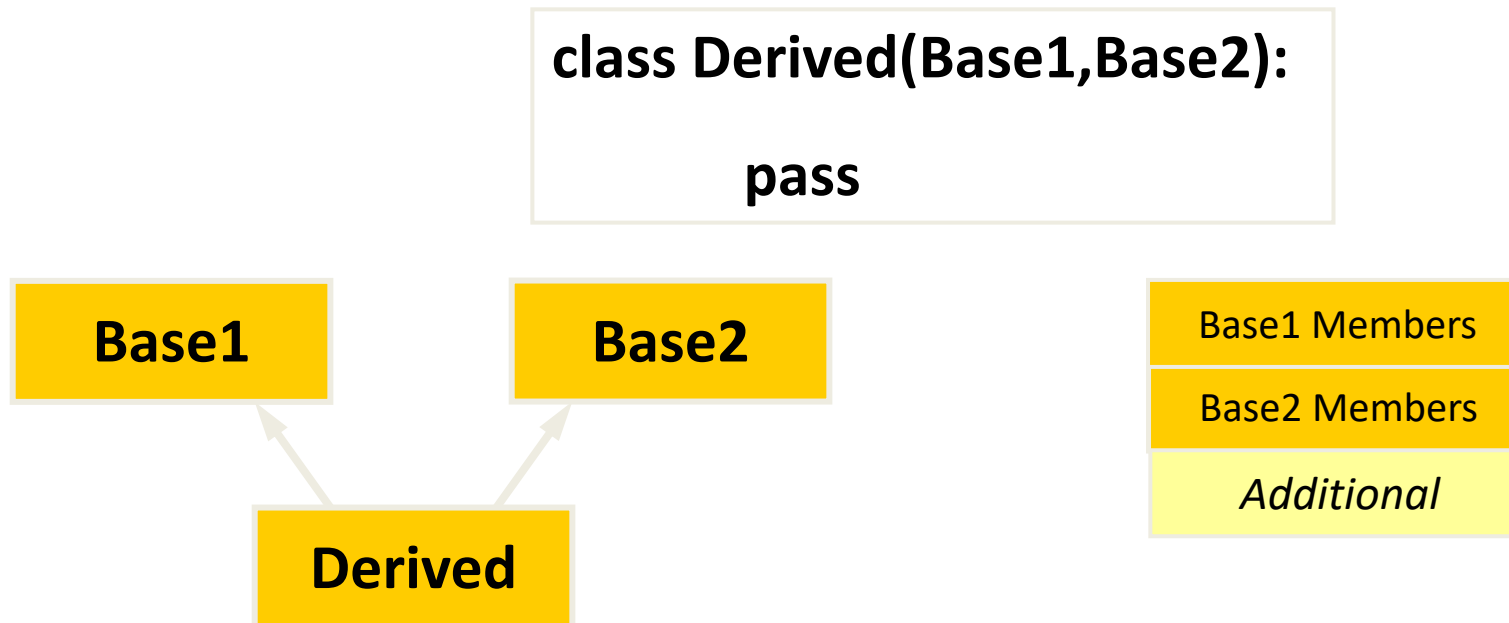
```
Starting engine Engine for car Car
engine started
Starting engine V8Engine for car RaceCar
engine started
Starting engine ElectricEngine for car CityCar
engine started
Starting engine V8Engine for car F1Car
engine started
```

```
car = Car(Engine())
racecar = RaceCar(V8Engine())
citycar = CityCar(ElectricEngine())
f1car = F1Car(V8Engine())
cars = [car, racecar, citycar, f1car]
for car in cars:
    car.start()
```

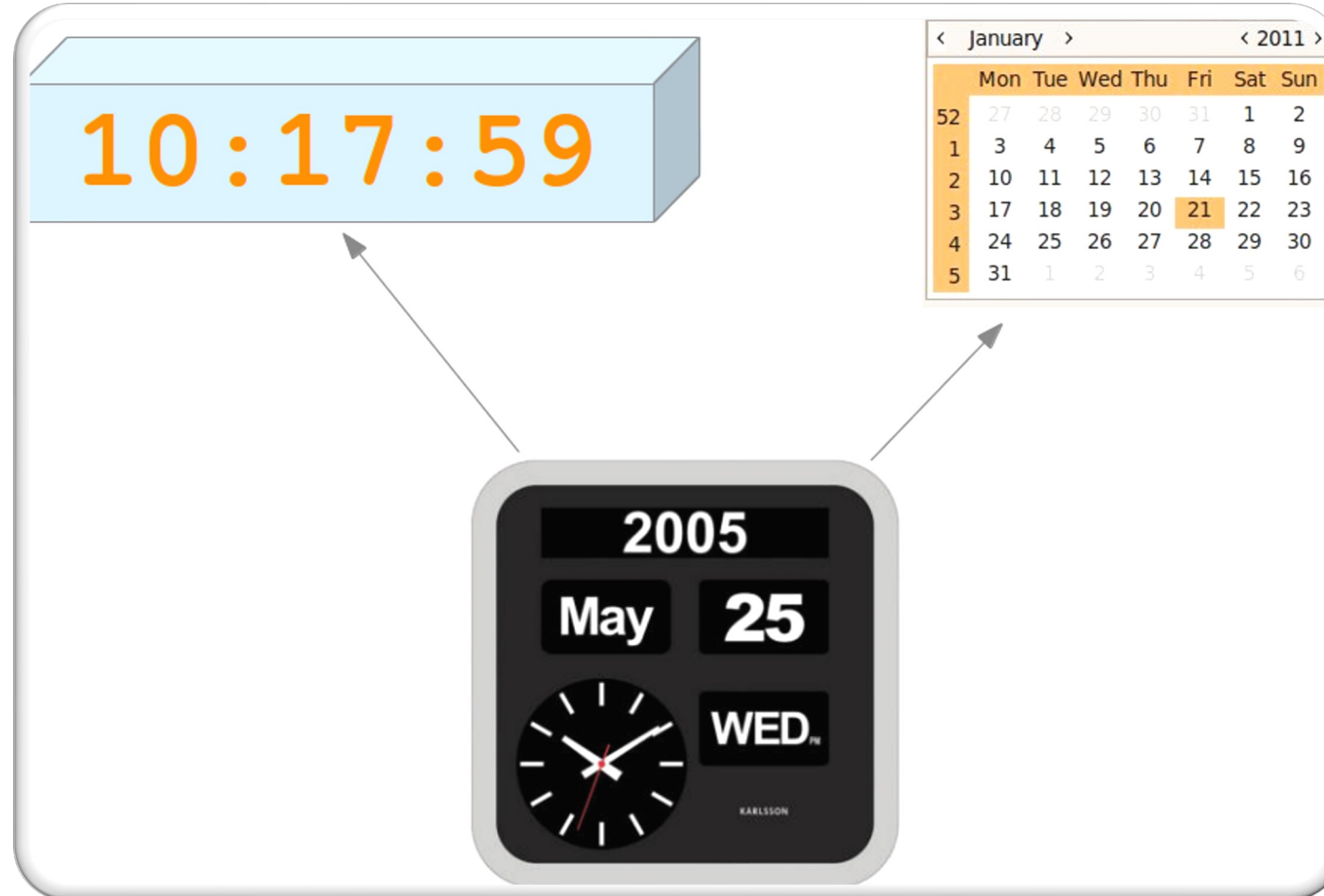
Multiple inheritance

- There are cases when we wish to inherit the characteristics of **more than one class**. That is, we wish to have more than one class as the base class.
- Inheriting more than one class is permitted in Python.
- This mechanisms is called: **Multiple Inheritance** !

(In General, multiple inheritance should be used wisely. There are cases in which we will use other techniques to avoid multiple inheritance.)



Motivation



Who is called first?

```
class A:  
    def m(self):  
        print("m of A called")
```

```
class B(A):  
    def m(self):  
        print("m of B called")
```

```
class C(A):  
    def m(self):  
        print("m of C called")
```

```
class D(B,C):  
    pass  
  
d1=D()  
d1.m()    #m of B is called
```

```
class D(C,B):  
    pass  
  
d1=D()  
d1.m()    #m of C is called
```

Who is called first?

```
class A:  
    def m(self):  
        print("m of A called")
```

```
class B(A):  
    pass
```

```
class C(A):  
    def m(self):  
        print("m of C called")
```

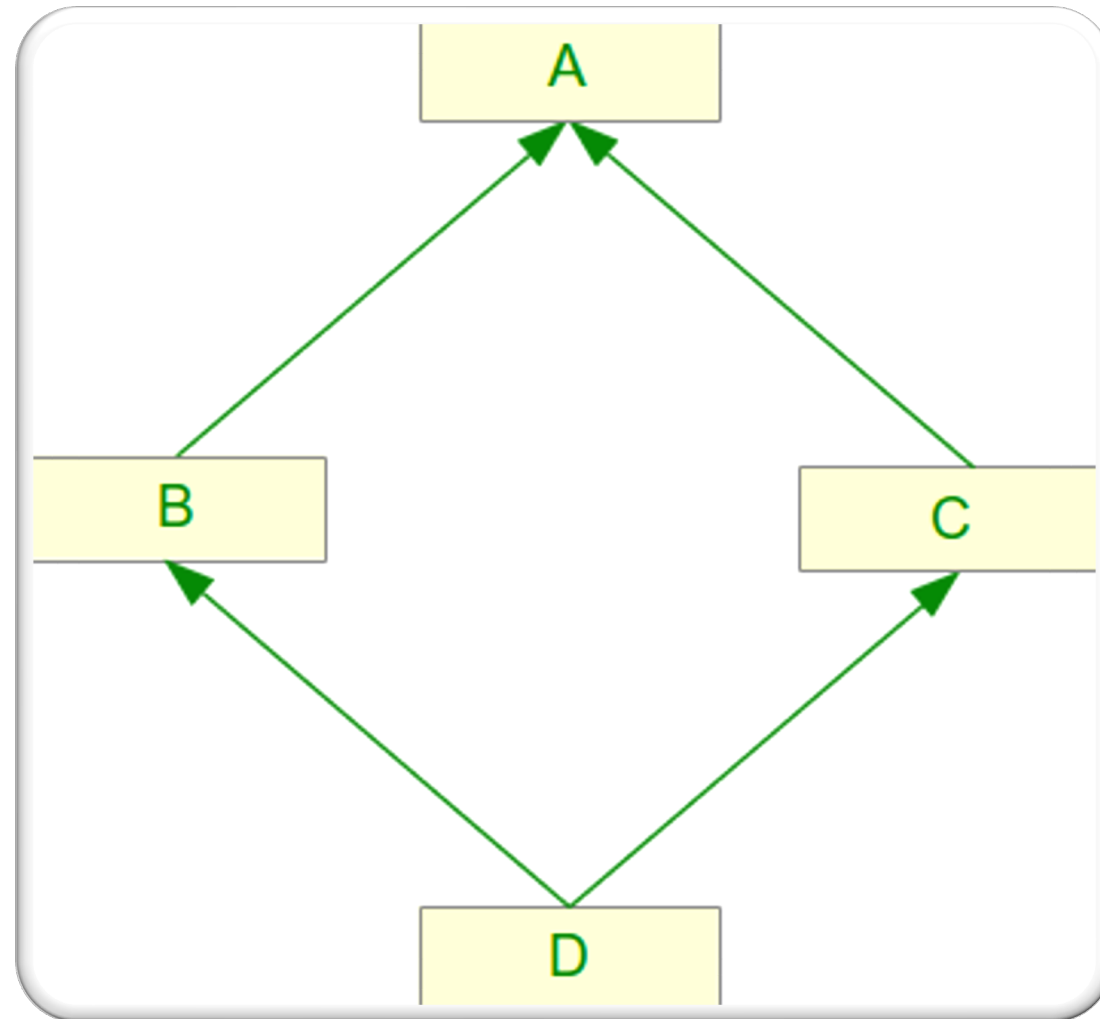
```
class D(B,C):  
    pass
```

```
x = D()  
x.m()
```

```
#in python 2.7: m of A is called  
#in python 3: m of C is called
```



The Diamond problem



Method resolution order (MRO)

- when you ask for *someobject.attribute*, and *attribute* is not found on that *object*, Python starts searching in *the class someobject* was created from.
- If it's not there either, Python searches up the inheritance chain (but how?).
- Python provides a way to always know what is the order in which classes are searched on attribute lookup: **the method resolution order**.

MRO in action

```
class A:  
    def m(self):  
        print("m of A called")
```

```
class B(A):  
    pass
```

```
class C(A):  
    def m(self):  
        print("m of C called")
```

```
class D(B,C):  
    pass  
  
x = D()  
print(x.__class__.__mro__)  
OR  
print(D.mro())
```

Output:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,  
<class '__main__.A'>, <class 'object'>]
```

MRO search illustration

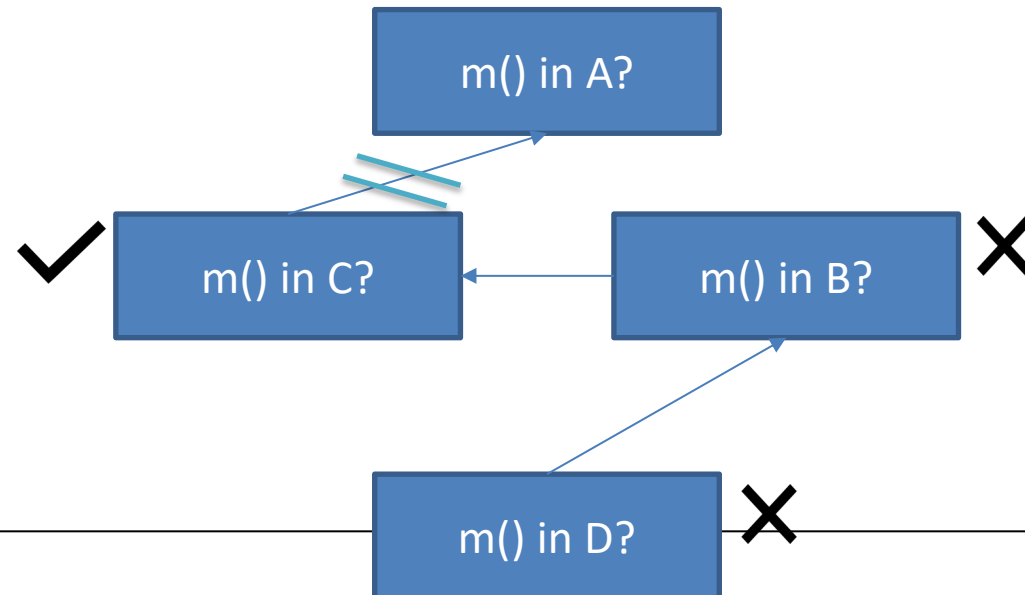
```
class A:
    def m(self):
        print("m of A called")
```

```
class B(A):
    pass
```

```
class C(A):
    def m(self):
        print("m of C called")
```

```
class D(B,C):
    pass

x = D()
print(x.m())
```



Calling INIT via class name

```
class A:
    def __init__(self):
        print("m of A called")
```

```
class B(A):
    def __init__(self):
        print("m of B called")
        A.__init__(self)
```

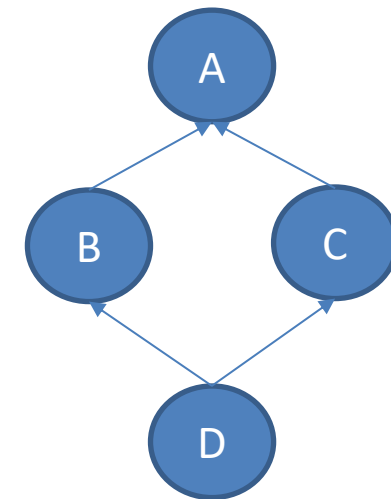
```
class C(A):
    def __init__(self):
        print("m of C called")
        A.__init__(self)
```

```
class D(B,C):
    def __init__(self):
        print("m of D called")
        B.__init__(self)
        C.__init__(self)
```

```
d1=D()
```

Output:

```
m of D called
m of B called
m of A called
m of C called
m of A called
```



Problematic...

Calling INIT via self

```
class A:
    def __init__(self):
        print("m of A called")
```

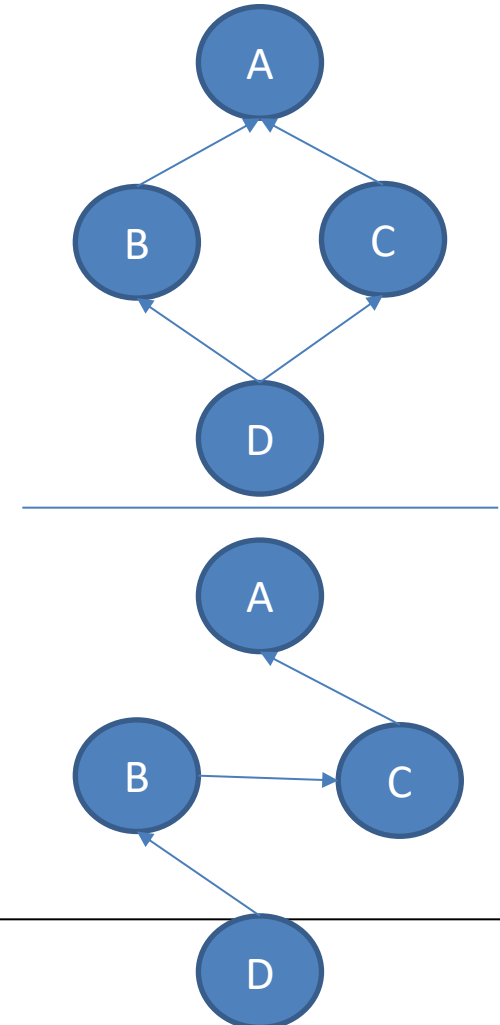
```
class B(A):
    def __init__(self):
        print("m of B called")
        super().__init__()
```

```
class C(A):
    def __init__(self):
        print("m of C called")
        super().__init__()
```

```
class D(B,C):
    def __init__(self):
        print("m of D called")
        super().__init__()
```

```
d1=D()
```

Output:
m of D called
m of B called
m of C called
m of A called



Private & public

DOUBLE UNDERSCORE “__”
NAME MANGLING



Encapsulation

- In Python, there is no keywords like 'public', 'protected' and 'private' to define the accessibility.
- In other words, in Python, it accepted that all attributes are public.
- But there is a method in Python to define Private: Add “__” in front of the variable and function name can hide them when accessing them from out of class.
- The mechanism is called “**name mangling**”

Private

```
class MyPrivate:

    def __init__(self):
        self.__privateVal = 0

    def adder(self):
        self.__privateVal += 1
        print (self.__privateVal)

p1 = MyPrivate()
p1.adder()
print(p1.__privateVal) #error
print(dir(p1))
```


Protecting the data

```
# Basic method of setting and getting attributes in Python
class Celsius:
    def __init__(self, temperature=0):
        self.temperature = temperature

    def to_fahrenheit(self):
        return (self.temperature * 1.8) + 32

# Create a new object
human = Celsius()

# Set the temperature
human.temperature = 37

# Get the temperature attribute
print(human.temperature)

# Get the to_fahrenheit method
print(human.to_fahrenheit())
```

Option 1: getters & setters

```
# Making Getters and Setter methods
class Celsius:
    def __init__(self, temperature=0):
        self.set_temperature(temperature)

    def to_fahrenheit(self):
        return (self.get_temperature() * 1.8) + 32

    # getter method
    def get_temperature(self):
        return self._temperature

    # setter method
    def set_temperature(self, value):
        if value < -273.15:
            raise ValueError("Temperature below -273.15 is not possible.")
        self._temperature = value

# Create a new object, set_temperature() internally called by __init__
human = Celsius(37)

# Get the temperature attribute via a getter
print(human.get_temperature())

# Get the to_fahrenheit method, get_temperature() called by the method itself
print(human.to_fahrenheit())

# new constraint implementation
human.set_temperature(-300)

# Get the to_fahreheit method
print(human.to_fahrenheit())
```

Option 1: getters & setters

- Problem - all the programs that implemented our previous class must modify their code from `obj.temperature` to `obj.get_temperature()` and all expressions like `obj.temperature = val` to `obj.set_temperature(val)`.
- This refactoring can cause problems while dealing with hundreds of thousands of lines of codes.
- All in all, our new update was not backwards compatible. This is where `@property` comes to rescue.

Option 2: property decorator

- Python programming provides us with a built-in `@property` decorator which makes usage of getter and setters much easier in Object-Oriented Programming.
- When you decorate a method with property, you can use the name of the method as if it was a data attribute.
- A method which is used for getting a value is decorated with `"@property"`
- The method which function as the setter is decorated with `"@x.setter"`. If the function had been called "f", we would have to decorate it with `"@f.setter"`.

Option 2: property decorator

```
# using property class
class Celsius:
    def __init__(self, temperature=0):
        self.temperature = temperature

    def to_fahrenheit(self):
        return (self.temperature * 1.8) + 32

    # getter
    def get_temperature(self):
        print("Getting value...")
        return self._temperature

    # setter
    def set_temperature(self, value):
        print("Setting value...")
        if value < -273.15:
            raise ValueError("Temperature below -273.15 is not possible")
        self._temperature = value

    # creating a property object
    temperature = property(get_temperature, set_temperature)
```

```
human = Celsius(37)

print(human.temperature)

print(human.to_fahrenheit())

human.temperature = -300
```

```
Setting value...
Getting value...
37
Getting value...
98.60000000000001
Setting value...
Traceback (most recent call last):
  File "C:/Users/inonz/OneDrive/Desktop/ex1.py", line 31, in <module>
    human.temperature = -300
  File "C:/Users/inonz/OneDrive/Desktop/ex1.py", line 18, in set_temperature
    raise ValueError("Temperature below -273.15 is not possible")
ValueError: Temperature below -273.15 is not possible
>>>
```

The @property Decorator

- In Python, `property()` is a built-in function that creates and returns a property object. The syntax of this function is:

```
property(fget=None, fset=None, fdel=None, doc=None)
```

where,

- `fget` is function to get value of the attribute
- `fset` is function to set value of the attribute
- `fdel` is function to delete the attribute
- `doc` is a string (like a comment)

Even shorter version

```
# Using @property decorator
class Celsius:
    def __init__(self, temperature=0):
        self.temperature = temperature

    def to_fahrenheit(self):
        return (self.temperature * 1.8) + 32

    @property
    def temperature(self):
        print("Getting value...")
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        print("Setting value...")
        if value < -273.15:
            raise ValueError("Temperature below -273 is not possible")
        self._temperature = value

# create an object
human = Celsius(37)

print(human.temperature)

print(human.to_fahrenheit())

coldest_thing = Celsius(-300)
```